



Autopent AI

Autopent AI System

Under supervision of:
Dr. Ahmed Gaber Abuabdallah

2025/2026

Team Members:

#	Name	Department	Mobile	Email
1	Malak Abdelaziz Mostafa	Network	01002470861	malak_a01658@cic-cairo.com
2	Nada Mohamed ELsayed	Network	01146600090	nada_a01763@cic-cairo.com
3	Rania Omar Aly	Network	01114217991	rania_a01425@cic-cairo.com
4	Bishoy Gerges Iskandar	Network	01277138421	bishoy_i01710@cic-cairo.com
5	Mohamed Ashraf Mohamed	Network	01116053498	mohamed_m01166@cic-cairo.com
6	Mahmoud Hamdy Khedr	Network	01100801691	mahmoud_k01184@cic-cairo.com
7	Abdelrahman Gamal Soliman	Network	01091419479	abdelrahman_m015322@cic-cairo.com
8	Mohamed Hussien Hassan	IS	01203591644	mohamed_h11237@cic-cairo.com

ACKNOWLEDGMENT

First and foremost, we would like to express our deepest gratitude and sincere appreciation to our supervisor, **Dr. Ahmed Gaber Abuabdullah**, for his continuous support, invaluable guidance, encouragement, and constructive feedback throughout the development of this graduation project. His expertise, patience, and dedication played a significant role in helping us overcome challenges and achieve the objectives of this work. We are truly grateful for the time and effort he devoted to reviewing our progress, providing insightful recommendations, and inspiring us to maintain high academic and professional standards. His guidance not only contributed to the successful completion of this project but also enriched our knowledge and understanding of the field. We would also like to extend our thanks to the faculty members of our department for providing us with the educational foundation and resources necessary for carrying out this project. Their support and dedication have been instrumental in our academic journey. Finally, we would like to express our heartfelt appreciation to our families and friends for their continuous encouragement, patience, and unwavering support throughout our studies and during the completion of this project. Their motivation and belief in our abilities have been a constant source of strength.

Special Dedication

We would like to specially dedicate our work to our parents, family and classmates for standing in our backs, encouraging us and guide us through different levels of our education and above all, we thank ALLAH for his blessings and granting us with this chance to improve ourselves and developing our minds to become better in our lives technically and socially.

May ALLAH keep his blessings on us and guide us from success to another.

Contents

Contents.....	1
Abstract	3
List of Figures	3
List of Tables.....	4
Chapter One: Introduction.....	1
1.1 Problem Definition	1
1.2 Problems in the existing system	1
1.3 The proposed Information System.....	1
1.4 Objectives	2
1.5 Methodology (System Development Approach (SDLC))	3
1.6 Project organization	3
Chapter Two: Planning Phase.....	4
2.1 Introduction.....	4
2.2 Problem Scope	4
2.3 Project schedule.....	4
2.3.1 Gantt Chart.....	5
2.4 Staff the project.....	7
2.5 Feasibility Study\.....	8
2.5.1 TYPES of Feasibility Study.....	8
2.6 PROJECT FEASIBILITY Study.....	8
2.6.1 Project Cost:.....	8
2.6.2 The Benefit:	8
2.6.3 Equilibrium (Breakeven) point:	9
2.7 SWOT Analysis	9
Chapter Three: System Analysis.....	10
3.1 Definition and Importance of System Analysis	10
3.2 Data Collection and Requirements Gathering	10
3.3 Tools	10
3.3.1 Analysis and Documentation Tools.....	10
3.3.3 AI and backend supporting Tools.....	12
3.4 Project Requirements.....	12
3.5 Data Flow Diagrams (DFD)	14
3.6 Use Case Diagram	15

Chapter Four: System Design.....	16
4.2 Entity Relationship Diagram (ERD).....	17
4.3 Class Diagram.....	18
4.4 Sequence Diagram	19
4.5 Activity Diagram	20
Chapter Five: System Implementation	21
5.1 Used programs	21
5.2 Screenshots of system.....	22
5.3 Codes of the important project modules	24
Chapter Six: System Testing.....	27
6.1 Types of Testing	27
6.1.1 Unit Testing	27
6.1.2 Integration Testing.....	27
6.1.3 Acceptance Testing.....	28
Chapter Seven: Conclusion and Future Work	29
7.1 Conclusion	29
7.2 Future Work.....	29
III-References.....	30

Abstract

With the rapid increase in cyber threats and the evolving digital landscape, ensuring web application security has become a critical challenge for organizations and developers. Traditional manual penetration testing is often time-consuming, resource-intensive, and requires specialized expertise, while existing automated tools typically suffer from fragmentation and complexity. These tools often generate passive technical reports that highlight risks without offering clear, actionable solutions, leaving systems vulnerable to exploitation for longer periods.

This project presents "AutoPent AI," an automated Vulnerability Scanner and Remediation Tool designed to bridge the gap between vulnerability detection and code correction. The system features a responsive, user-friendly interface developed using React.js, coupled with a robust backend controller built in Python to orchestrate complex security operations. The core engine seamlessly integrates powerful open-source security standards, including OWASP ZAP for web application scanning, Nmap for network port analysis, and OpenVAS for infrastructure vulnerability management, into a single efficient engine.

A key innovation of this system is its AI-driven Remediation Module. Unlike traditional scanners, this system utilizes artificial intelligence to analyze vulnerability findings and generate specific, syntax-correct code fixes and patching suggestions. Developed following the Agile SDLC methodology to allow for iterative improvements and continuous feedback, this approach transforms the penetration testing process from a passive reporting task into an active, intelligent defense mechanism, enabling developers to secure their applications efficiently and effectively.

List of Figures

Figure 3.1 Context Data Flow Diagram (DFD)	14
Figure 3.2 Use Case Diagram.....	15
Figure 4.1 System Architecture Diagram.....	16
Figure 4.2 Entity Relationship Diagram (ERD)	17
Figure 4.3 Class Diagram.....	18
Figure 4.4 Sequence Diagram.....	19
Figure 4.5 Activity Diagram.....	20
Figure 5.1 Home Page	22
Figure 5.2 Entry Page	22
Figure 5.3 Vulnerability Report	23
Figure 5.4 URL Validation Module	24
Figure 5.5 Vulnerability Scanner Module	24
Figure 5.6 Report Generation Module	25
Figure 5.7 Database Connection Module	25
Figure 5.8 Authentication	26

List of Tables

Table 2.1: Gantt Chart.....	5
Table 2.2: Swot Analysis	9
Table 6.1: System Testing Results	28
Table 7.1: Future Work Plan.....	29

Chapter One: Introduction

1.1 Problem Definition

In today's rapidly evolving digital landscape, web applications and network infrastructures are increasingly vulnerable to cyber threats, including injection attacks, misconfigurations, and unauthorized access. Developers often struggle to secure their applications effectively because manual penetration testing is time-consuming, expensive, and requires specialized expertise. The core problem is not just detecting vulnerabilities—such as open ports, weak configurations, or injection flaws—but bridging the gap between detection and remediation. Existing tools often generate complex technical reports without providing developers with clear, actionable code fixes, leaving systems vulnerable to exploitation. Existing tools typically produce technical reports that highlight risks and evidence but fail to provide developers with clear, automated solutions, leaving systems exposed to exploitation and increasing the risk of breaches. Consequently, organizations face a critical need for a solution that not only identifies these risks efficiently but also accelerates the remediation process to minimize the window of exposure before attackers can exploit them.

1.2 Problems in the existing system

Current security practices and automated scanners face several limitations:

- **Fragmentation:** Security teams often rely on multiple disconnected tools (e.g., one for network scanning, another for web vulnerabilities), making the process inefficient.
- **Lack of Remediation:** Most scanners provide a list of risks and evidence (the "what") but fail to suggest specific code corrections (the "how").
- **Complexity:** Generated reports are often too technical for standard developers to interpret and act upon quickly.
- **Manual Effort:** The process of verifying vulnerabilities and researching fixes manually is slow and error-prone to human error.

1.3 The proposed Information System

The proposed project is a comprehensive Vulnerability Scanner and AI-Driven Remediation Tool designed to modernize the security testing lifecycle. It acts as a unified platform that not only automates the detection of vulnerabilities across web applications and networks but also bridges the gap to remediation by providing intelligent, AI-driven code fixes, empowering developers to resolve security issues instantly. Furthermore, the system provides a centralized dashboard that enables users to monitor scan progress, review detected vulnerabilities, and access detailed security reports. Combining automated vulnerability assessment tools with artificial intelligence capabilities, the platform significantly reduces the time and effort required for security testing and remediation. The proposed system also enhances decision-making by prioritizing vulnerabilities based on their severity and providing actionable recommendations to improve the overall security posture of web applications and network infrastructures.

- **System Workflow:**

Input: The user (Developer/Admin) enters the target URL or IP address.

Scanning Engine: The backend leverages powerful open-source tools—**Burp Suite** for web/API vulnerabilities, **Nmap** for network port scanning, and **OpenVAS** for vulnerability management—to perform automated scans.

Analysis and Reporting: Scan results are analyzed to calculate a risk score, compile detailed evidence, and generate a report.

The system offers three key features:

Basic Report: Displays only the risk score and high-level evidence for quick overviews. **Auto-Fix Recommendations:** Feeds the report to an AI tool to generate specific code fixes or configuration changes.

Interactive AI Assistance: Provides a chat interface for querying the AI about Vulnerabilities, fixes, or security advice.

Technical Architecture: The system is built using a decoupled architecture:

Frontend: Developed using **ReactJS** to ensure a responsive and user-friendly interface.

Backend: Built with **Python (Django/Flask)** or **node.js** to act as the controller, managing the scanning tools and processing data.

The system is designed strictly for authorized penetration testing and defensive security purposes, with safeguards like user authentication.

1.4 Objectives

The main objectives of this project are:

1. To develop an automated web-based vulnerability scanner using **PHP, JavaScript, and Python**.
2. To integrate industry-standard security tools (Burp Suite, Nmap, OpenVAS) into a single efficient engine.
3. To bridge the gap between security and development by providing **AI-generated code fixes** for detected vulnerabilities.
4. To implement a secure user environment with authentication (Login/Signup) and a history log to track previous scans.
5. To produce a working prototype that demonstrates local scanning capabilities, with plans to expand to advanced AI chat features in the second phase.
6. To ensure ethical and legal usage by limiting scans to authorized targets only.

1.5 Methodology (System Development Approach (SDLC))

The project follows the **Agile SDLC methodology**, which supports iterative development and flexibility. This approach allows the team to adapt to new requirements and technical challenges that may arise during the implementation of complex security tools. The development process is divided into managed sprints, ensuring continuous testing and feedback integration. This is particularly suitable for the project's two-term structure:

- **Phase 1 (Current Term):** Focuses on tool research, prototype development, core scanning engine implementation, and basic frontend-backend integration.
- **Phase 2 (Next Term):** Focuses on integrating the advanced AI Remediation Module, implementing the chat feature, and finalizing the system for deployment.

1.6 Project organization

The documentation is organized to cover the full lifecycle of the project:

- **Chapter 1:** Introduction, defining the problem, proposed solution, and objectives.
- **Chapter 2:** Planning Phase, outlining the project schedule (Gantt Chart) and feasibility study.
- **Chapter 3:** System Analysis, detailing functional requirements, Data Flow Diagrams (DFD), and Use Cases.
- **Chapter 4:** System Design, covering the database structure (ERD) and system architecture.
- **Chapter 5:** System Implementation, describing the development tools and showing prototype screenshots.
- **Chapter 6:** System Testing, including unit and integration testing.
- **Chapter 7:** Conclusion and Future Work.

Chapter Two: Planning Phase

2.1 Introduction

The planning phase is a fundamental step in the software development life cycle. It establishes the roadmap for the project, ensuring that the development process is organized, efficient, and aligned with the project's objectives. In this chapter, we define the scope of the "Vulnerability Scanner and AI-Driven Remediation Tool," outline the team structure, and present a comprehensive feasibility study to verify the project's viability.

2.2 Problem Scope

Defining the scope helps in focusing the development efforts on the core features required for the prototype and the final product.

- **In-Scope:**

Developing a responsive web interface using **React.js** for user interaction.

Building a robust backend controller using **Python** to manage scanning operations.

Integrating open-source security tools (**Burp Suite** for web apps, **Nmap** for network ports, and **OpenVAS** for vulnerability management).

Generating automated reports that include risk levels (High, Medium, Low) and detailed evidence.

Implementing an **AI-based Remediation Module** to suggest code fixes for detected vulnerabilities automatically.

User management features including secure Registration, Login, and Scanning History Logs.

- **Out-of-Scope:**

Exploitation: The tool detects vulnerabilities but does not perform active attacks or exploitation (it is a defensive tool).

Automatic Patching: The system provides code suggestions, but it does not automatically modify or write to the target server's source code files to prevent accidental damage.

Scanning Unauthorized Targets: The tool is intended for authorized testing only.

2.3 Project schedule

The project schedule presents the sequence of phases and tasks required to develop the system in an organized manner.

The project follows the Software Development Life Cycle (SDLC), starting with planning and analysis, followed by system design, implementation, testing, and documentation.

The timeline is divided into two main phases corresponding to the academic terms.

A Gantt chart is used to visualize task durations, dependencies, and key milestones throughout the project.

2.3.1 Gantt Chart

Table 2.1: Gantt Chart

ID	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Timeline											
							1st Quarter		2nd Quarter		3rd Q							
							Nov	Dec	Jan	Feb	Mar	Apr	May	Jun	Jul			
1	PHASE 1: PLANNING & RESEARCH	18 days	Mon 11/3/25	Wed 11/26/25		All Team												
2	Project Team Formation & Roles	1 day	Mon 11/3/25	Mon 11/3/25		Malak Abdelaziz												
3	Setup Communication	1 day	Tue 11/4/25	Tue 11/4/25	2	Bishoy Gerges												
4	Define Project Scope & Objectives	2 days	Wed 11/5/25	Thu 11/6/25	3	Malak,Bishoy												
5	Identify Key Stakeholders	1 day	Fri 11/7/25	Fri 11/7/25	4	Malak Abdelaziz												
6	Conduct Feasibility Study	2 days	Fri 11/7/25	Mon 11/10/25	5	Abdelrahman,Mahmoud												
7	Research Web Scanners (ZAP)	6 days	Tue 11/11/25	Tue 11/18/25	6	Bishoy,Mohamed Hussein												
8	Research Network Scanners (Nmap)	6 days	Tue 11/11/25	Tue 11/18/25	6	Mahmoud,Rania Omar												
9	Research AI Models for Fixes	6 days	Tue 11/11/25	Tue 11/18/25	6	Mohamed Ashraf,Nada												
10	Compare & Select Tech Stack	2 days	Wed 11/19/25	Thu 11/20/25	7,8,9	Bishoy,Abdelrahman												
11	Define Hardware/Software Req.	1 day	Fri 11/21/25	Fri 11/21/25	10	Nada,Rania												
12	Draft Project Proposal	2 days	Mon 11/24/25	Tue 11/25/25	11	Malak,Mohamed Hussein												
13	Review & Submit Proposal	1 day	Wed 11/26/25	Wed 11/26/25	12	Malak Abdelaziz												
14	PHASE 2: ANALYSIS (TERM 1)	23 days	Thu 11/27/25	Mon 12/29/25		Analysts												
15	Gather Functional Requirements	4 days	Thu 11/27/25	Tue 12/2/25	13	Malak,Rania Omar												
16	Analyze Non-Functional Requirements	2 days	Wed 12/3/25	Thu 12/4/25	15	Abdelrahman,Mahmoud												
17	Perform Risk Analysis	2 days	Fri 12/5/25	Mon 12/8/25	16	Malak,Bishoy												
18	Define User Roles & Permissions	1 day	Wed 12/3/25	Wed 12/3/25	17	Bishoy Gerges												
19	Define AI Model Data Sources	3 days	Wed 12/3/25	Fri 12/5/25	17	Mohamed Ashraf,Nada												
20	Create Context Diagram (DFD L0)	2 days	Thu 12/4/25	Fri 12/5/25	18	Mohamed Hussein												
21	Create DFD Level 1 (Detailed)	3 days	Mon 12/8/25	Wed 12/10/25	20	Mohamed Hussein,Nada												
22	Create Use Case Diagrams	2 days	Thu 12/4/25	Fri 12/5/25	18	Rania,Mohamed Ashraf												
23	Create Functional Requirements Doc (SRS)	4 days	Mon 12/8/25	Thu 12/11/25	22	Mohamed Hussein												
24	Write Use Case Scenarios	3 days	Mon 12/8/25	Wed 12/10/25	22	Rania,Malak												
25	PHASE 3: DESIGN (CRITICAL FOR)	13 days	Thu 12/11/25	Mon 12/29/25		Designers												
26	Design Conceptual Data	2 days	Thu 12/11/25	Fri 12/12/25	24	Mahmoud,Bishoy												
27	Design ERD (Database)	3 days	Mon 12/15/25	Wed 12/17/25	26	Mahmoud,Abdelrahman												
28	Design System Architecture	2 days	Thu 12/18/25	Fri 12/19/25	27	Bishoy Gerges												
29	Design Network Topology (Nmap)	3 days	Mon 12/22/25	Wed 12/24/25	28	Mahmoud Hamdy												
30	Design Security Architecture	3 days	Thu 12/25/25	Mon 12/29/25	29	Bishoy,Abdelrahman												
31	Design Class Diagrams	3 days	Thu 12/18/25	Mon 12/22/25	27	Abdelrahman Gamal												
32	Design Sequence Diagrams	3 days	Thu 12/18/25	Mon 12/22/25	27	Nada Mohamed												
33	Design UI Wireframes (Login/Reg)	3 days	Thu 12/11/25	Mon 12/15/25	24	Rania,Mohamed Hussein												
34	Design Dashboard	3 days	Tue 12/16/25	Thu 12/18/25	33	Bishoy,Rania Omar												
35	Design Report Interface	2 days	Fri 12/19/25	Mon 12/22/25	34	Mohamed Hussein												
36	Design UI Style Guide	2 days	Tue 12/23/25	Wed 12/24/25	39	Rania Omar												
37	PHASE 4: DOCS WRITING (Ch 1, 2, 3, 4)	34 days	Thu 11/27/25	Tue 1/13/26		All (Split)												
38	Write Chapter 1 (Introduction)	5 days	Thu 11/27/25	Wed 12/3/25	13	Malak,Nada Mohamed												
39	Write Chapter 2 (Planning)	5 days	Thu 11/27/25	Wed 12/3/25	13	Abdelrahman,Mahmoud												

ID	Task Name	Duration	Start	Finish	Predecessors	Resource Names	1st Quarter	2nd Quarter	3rd Q
							Jan	Feb	Mar
40	Write Chapter 3 (Analysis)	5 days	Tue 12/30/25	Mon 1/5/26	23,24	Malak,Mohamed Ashraf			
41	Write Chapter 4 (System Design)	7 days	Tue 12/30/25	Wed 1/7/26	30,32,30,29,29	Nada,Rania			
42	Review All Chapters (1, 2, 3, 4)	4 days	Thu 1/8/26	Tue 1/13/26	38,39,40,41	Malak,Bishoy			
43	SUBMIT DOCUMENTATION (CH 1-4)	0 days	Tue 1/13/26	Tue 1/13/26	42	All Team			
44	PHASE 5: FRONTEND PROTOTYPE	27 days	Fri 12/12/25	Mon 1/19/26		Frontend Team			
45	Setup Git Repository for	1 day	Fri 12/12/25	Fri 12/12/25	10,19,23	Bishoy Gerges			
46	Create Project Structure	1 day	Mon 12/15/25	Mon 12/15/25	49	Mohamed Hussein			
47	Develop Global Navigation Bar	2 days	Tue 12/16/25	Wed 12/17/25	48	Rania Omar			
48	Develop Footer Component	1 day	Thu 12/18/25	Thu 12/18/25	47	Mohamed Hussein			
49	Develop Registration Page	3 days	Thu 12/18/25	Mon 12/22/25	47	Mohamed Hussein			
50	Develop Login Page	3 days	Thu 12/18/25	Mon 12/22/25	47	Rania Omar			
51	Develop Dashboard Sidebar Menu	2 days	Tue 12/23/25	Wed 12/24/25	50	Bishoy Gerges			
52	Develop "New Scan" Input Form	3 days	Thu 12/25/25	Mon 12/29/25	51	Mohamed Hussein			
53	Develop "Scan History" Data Table	3 days	Thu 12/25/25	Mon 12/29/25	51	Rania Omar			
54	Implement Static Charts (Chart.js)	3 days	Tue 12/30/25	Thu 1/1/26	53	Rania,Bishoy			
55	Refine UI: Make Responsive	5 days	Fri 1/2/26	Thu 1/8/26	54	Rania,Mohamed Hussein			
56	Deliver UI Prototype	0 days	Thu 1/8/26	Thu 1/8/26	55	Frontend Team			
57	Mid-Year Exams Break (Limited Work on Docs)	18 days	Thu 12/25/25	Mon 1/19/26		-			
58	PHASE 6: BACKEND DEV (TERM 2)	34 days	Mon 1/19/26	Thu 3/5/26		Backend Team			
59	Setup Python Flask Virtual Env	1 day	Tue 1/20/26	Tue 1/20/26	61	Mahmoud Hamdy			
60	Install Project Dependencies	1 day	Wed 1/21/26	Wed 1/21/26	59	Abdelrahman Gamal			
61	Configure Git for Backend	1 day	Mon 1/19/26	Mon 1/19/26	43	Bishoy Gerges			
62	Initialize MySQL Database	2 days	Thu 1/22/26	Fri 1/23/26	60	Abdelrahman Gamal			
63	Module 1: Authentication & Authorization APIs	4 days	Mon 1/26/26	Thu 1/29/26	62	Mahmoud Hamdy			
64	Develop User Profile APIs	3 days	Fri 1/30/26	Tue 2/3/26	63	Abdelrahman Gamal			
65	Module 2: Scanner Integration (Nmap/ZAP)	8 days	Mon 1/26/26	Wed 2/4/26	62	Bishoy,Mahmoud			
66	Script: OWASP ZAP API Client	8 days	Mon 1/26/26	Wed 2/4/26	62	Abdelrahman,Nada			
67	Script: OpenVAS Scanner Integration	8 days	Mon 1/26/26	Wed 2/4/26	62	Nada,Mohamed Hussein			
68	Implement Nmap Result Parser	4 days	Thu 2/5/26	Tue 2/10/26	69	Mahmoud Hamdy			
69	Implement ZAP Result Parser	4 days	Thu 2/5/26	Tue 2/10/26	68	Abdelrahman Gamal			
70	Develop Vulnerability Risk	4 days	Wed 2/11/26	Mon 2/16/26	68,69	Bishoy Gerges			
71	Create PDF Report Generation Logic	5 days	Tue 2/17/26	Mon 2/23/26	70	Abdelrahman Gamal			
72	Module 3: Logging & Reporting System	5 days	Tue 2/17/26	Mon 2/23/26	70	Abdelrahman,Nada			
73	Create Scan Request/Result	4 days	Tue 2/24/26	Fri 2/27/26	71	Mahmoud,Nada			
74	Backend Error Handling & Logging	3 days	Mon 3/2/26	Wed 3/4/26	73	Bishoy,Mahmoud			
75	Backend Code Review & Cleanup	1 day	Thu 3/5/26	Thu 3/5/26	74	Bishoy Gerges			
76	System Ready for Integration	0 days	Thu 3/5/26	Thu 3/5/26	75	Abdelrahman Gamal			

ID	Task Name	Duration	Start	Finish	Predecessors	Resource Names	Timeline																	
							Quarter	1st Quarter			2nd Quarter			3rd Quarter			4th Quarter			1st Q				
							Nov	Dec	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan			
77	PHASE 7: AI MODULE DEV (TERM 2)	30 days	Thu 2/19/26	Wed 4/1/26		AI Team																		
78	Collect Vulnerability	5 days	Thu 2/19/26	Wed 2/25/26	77	Mohamed Ashraf																		
79	Data Cleaning & Normalization	4 days	Thu 2/26/26	Tue 3/3/26	78	Nada Mohamed																		
80	Split Data (Train/Test/Validate)	2 days	Wed 3/4/26	Thu 3/5/26	79	Nada Mohamed																		
81	Select & Configure LLM Model	3 days	Fri 3/6/26	Tue 3/10/26	80	Mohamed Ashraf																		
82	Configure GPU Training Env	2 days	Wed 3/11/26	Thu 3/12/26	81	Mohamed Ashraf																		
83	Train AI Model (Fine-Tuning)	8 days	Fri 3/13/26	Tue 3/24/26	82	Mohamed Ashraf																		
84	Evaluate Model Accuracy	3 days	Wed 3/25/26	Fri 3/27/26	83	Malak,Mohamed Ashraf																		
85	Build AI Prediction API (Flask)	3 days	Mon 3/30/26	Wed 4/1/26	84	Mohamed Ashraf																		
86	PHASE 8: INTEGRATION & DEPLOYMENT	32 days	Thu 4/2/26	Fri 5/15/26		Integration																		
87	Connect Frontend to Auth APIs	3 days	Thu 4/2/26	Mon 4/6/26	79,83,88	Bishoy,Rania																		
88	Connect Dashboard to User Data	3 days	Tue 4/7/26	Thu 4/9/26	87	Mahmoud,Mohamed Hussein																		
89	Integrate AI API with Backend	4 days	Thu 4/2/26	Tue 4/7/26	89,79	Bishoy,Mohamed Ashraf																		
90	Connect Scan Forms to Engine	4 days	Wed 4/8/26	Mon 4/13/26	89	Bishoy,Mahmoud																		
91	Release Beta Version (Internal)	0 days	Mon 4/13/26	Mon 4/13/26	90	Malak Abdelaziz																		
92	Unit Testing (Frontend/Backend)	6 days	Tue 4/14/26	Tue 4/21/26	91	Rania,Abdelrahman																		
93	Integration Testing (End-to-End)	7 days	Wed 4/22/26	Thu 4/30/26	92	Bishoy,Mahmoud																		
94	Perform Security Penetration Test	7 days	Fri 5/1/26	Mon 5/11/26	93	Bishoy,Abdelrahman																		
95	Performance & Load Testing	4 days	Tue 5/12/26	Fri 5/15/26	94	Abdelrahman,Mo Hussein																		
96	Validate AI Remediation Suggestions	2 days	Tue 5/12/26	Wed 5/13/26	94	Mohamed Ashraf,Malak																		
97	PHASE 9: FINAL CLOSURE (CH 5, 6, 7)	20 days	Mon 5/18/26	Fri 6/12/26	98,99	All Team																		
98	Fix Bugs & Performance Issues	5 days	Mon 5/18/26	Fri 5/22/26	98,99	All Developers																		
99	Conduct User Acceptance Testing	3 days	Mon 5/25/26	Wed 5/27/26	98	Malak,Rania																		
100	Write Ch 5 (Implementation Details)	6 days	Thu 5/28/26	Thu 6/4/26	99	Mahmoud,Mohamed Ashraf																		
101	Write Ch 6 (Testing & Results)	4 days	Thu 5/28/26	Tue 6/2/26	99	Abdelrahman,Mo Hussein																		
102	Write Ch 7 (Conclusion & Recommendations)	2 days	Thu 5/28/26	Fri 5/29/26	99	Malak Abdelaziz																		
103	Create User Manual & Guides	3 days	Thu 5/28/26	Mon 6/1/26	99	Mohamed Hussein																		
104	Write Technical Documentation	5 days	Fri 6/5/26	Thu 6/11/26	100	Bishoy,Mahmoud																		
105	Create Installation Guide	2 days	Fri 6/5/26	Mon 6/8/26	100	Abdelrahman Gamal																		
106	Review & Format Final Thesis	4 days	Tue 6/2/26	Fri 6/5/26	103	Malak,Bishoy																		
107	Prepare Presentation & Slides	3 days	Mon 6/8/26	Wed 6/10/26	106	Rania,Mohamed Ashraf																		
108	Presentation Rehearsal	2 days	Thu 6/11/26	Fri 6/12/26	107	All Team																		
109	FINAL PROJECT DELIVERY	0 days	Fri 6/12/26	Fri 6/12/26	108	All Team																		

2.4 Staff the project

The project is developed by a collaborative team, where responsibilities are distributed based on technical expertise. The key roles include:

1. **Project manager:** Responsible for timeline management, task allocation, and documentation.
2. **Frontend Developer:** Responsible for designing and implementing the User Interface (UI) using **React.js**
3. **Backend Developer:** Responsible for the logic layer, building the **Python** controller, and managing API integration.
4. **Security Engineer:** Responsible for configuring the scanning tools (Burp suite, Nmap) and defining scan parameters.

5. **AI Engineer:** Responsible for integrating the AI model to generate remediation suggestions.
6. **Quality Assurance (Tester):** Responsible for testing the system functionalities and verifying scan accuracy.

2.5 Feasibility Study\

A feasibility study determines whether the project is practical and worth developing. We have analyzed the project from three main perspectives: Technical, Operational, and Economic.

2.5.1 TYPES of Feasibility Study

1. **Technical Feasibility:** Assesses if the available technical resources and skills are sufficient.
2. **Operational Feasibility:** Assesses if the system will solve the problem and be used effectively.
3. **Economic Feasibility:** Assesses the cost-benefit analysis.

2.6 PROJECT FEASIBILITY Study

2.6.1 Project Cost:

A. Development cost:

Labor: Zero monetary cost (Student effort).

Time: Estimated at approx. 600 man-hours over two terms.

B. Hardware cost:

Personal Laptops (Core i5/i7, 8GB+ RAM) – *Already available resources.*

Localhost Server for testing – *Zero cost.*

C. Software cost:

Languages: Python, PHP, React.js (Open Source / Free).

Tools: Burp Suite, Nmap, OpenVAS (Open Source / Free).

IDE: VS Code, PyCharm (Community Editions / Free).

D. Operational cost:

Electricity and Internet connectivity expenses during development.

2.6.2 The Benefit:

- **Tangible Benefits:**

Reduces the time required for penetration testing by automating the scanning process and reporting.

Provides instant, actionable code fixes, saving developers research time.

- **Intangible Benefits:**

Increases security awareness among developers.

Improves the security posture of web applications before deployment.

2.6.3 Equilibrium (Breakeven) point:

As an open-source student project with minimal financial investment, the project "breaks even" immediately upon deployment. Its value is measured in educational gain and the successful detection of vulnerabilities rather than financial profit.

2.7 SWOT Analysis

An analysis of the Strengths, Weaknesses, Opportunities, and Threats facing the project.

- **Strengths:**
 - Automation:** Combines detection and solution in one-step.
 - AI Integration:** Offers a competitive edge by suggesting specific code fixes.
 - Cost-Effective:** Uses entirely free and open-source tools.
- **Weaknesses:**
 - False Positives:** Automated scanners may sometimes report incorrect vulnerabilities.
 - Complexity:** Integrating multiple tools (Python + Security Tools) requires complex configuration.
- **Opportunities:**
 - Scalability:** Can be expanded to include Cloud Security scanning.
 - DevSecOps:** Potential for integration into CI/CD pipelines for software companies.
- **Threats:**
 - Evolving Threats:** New vulnerabilities appear daily, requiring constant updates to the scanner's database.
 - Legal Risks:** Potential misuse of the tool for malicious scanning if not properly secured.

Table 2.1: SWOT Analysis Summary

Table 1.2: Swot Analysis

Category	Description
Strengths	Automation, AI integration, and cost-effectiveness.
Weaknesses	False positives and system complexity.
Opportunities	Cloud security expansion and DevSecOps integration.
Threats	Evolving threats and legal risks.

Chapter Three: System Analysis

3.1 Definition and Importance of System Analysis

System analysis is the process of observing and analyzing systems to identify their objectives and determine how to build them efficiently. In this project, system analysis is crucial to bridge the gap between complex security requirements—such as vulnerability scanning and automated remediation—and the actual software implementation. It helps in understanding user needs, defining the scope, and preventing potential logical errors during the coding phase.

3.2 Data Collection and Requirements Gathering

To build an effective Vulnerability Scanner, we utilized several methods to gather requirements and understand the problem domain:

1. **Literature Review:** Researching the documentation of open-source security tools (OWASP ZAP, Nmap, OpenVAS) to understand their capabilities, APIs, and limitations.
2. **Competitor Analysis:** Analyzing existing vulnerability scanners to identify gaps in the market, specifically the lack of direct "AI-driven code fixes" in standard reports.
3. **Consultation:** Regular meetings with the project supervisor to define the boundaries of the prototype and the final product.

3.3 Tools

This section describes the tools used throughout the project. The tools are grouped based on their purpose into analysis tools, security tools, and AI and backend tools to ensure an organized and efficient development process

3.3.1 Analysis and Documentation Tools

During the analysis and design phase (before coding), the following tools were utilized to document and model the system:

- **Documentation Tools:** Microsoft Word or Google Docs for drafting the project report and maintaining records
- **Modeling & Diagramming:** Draw.io (or Microsoft Visio) for creating Data Flow Diagrams (DFD), Use Case Diagrams, and ERD.
- **Project Management Tools:** Trello/Jira for task tracking and organizing the agile sprint workflow.

3.3.2 Security Scanning Tools Research

- **Burp Suite:** Open source web application security scanner used to detect common vulnerabilities such as SQL injection, XSS, and insecure configurations.
- **How it works:** Burp Suite scans web pages and HTTP requests to identify insecure behaviours and vulnerabilities. Its API supports easy integration and automation.

Features:

- Paid
- Support automated scanning and report generation.

Limitations:

- Requires security knowledge to analyze results accurately.
- **NMAP** A network scanning tool used to identify open ports, running services, and potential network level vulnerabilities. Nmap is used to perform an initial network assessment before application-level scanning and to identify exposed services and suitable targets for further security testing.

How it works: NMAP uses network protocols to scan hosts and services within a network. It provides information about OS, service versions, and network configurations.

Features:

- Free and open-source.
- Fast and accurate network scanning.
- Provides detailed information about network topology.

Limitations:

- Does not detect web application vulnerabilities.
- Require technical knowledge to interpret scan results correctly.
- **OpenVAS** A vulnerability management framework used to detect known security weaknesses and provides detailed risk levels and technical evidence. OpenVAS is used to assess servers and network systems related to the project to ensure that no known vulnerabilities exist.

How it works: OpenVAS relies on a continuously updated vulnerability database to identify known vulnerabilities. It generates detailed reports that include risk levels and technical evidence.

Features:

- Free and open-source.
- Covers a wide range of known vulnerabilities.
- Provide risk classification to help prioritize vulnerabilities.

Limitations:

- Scan results may include false positives that require manual verification.

3.3.3 AI and backend supporting Tools

- **AI language Model:** Studied for analyzing vulnerability reports and generating human-readable remediation steps and code-level recommendations.
- **Backend Framework:** Python-based frameworks (flask/django) were evaluated for controlling scanning tools, processing results and communicating with the frontend via APIs.

3.4 Project Requirements

The system requirements are categorized into Functional (what the system does) and Non-Functional (how the system performs).

A. Functional Requirements These requirements define the specific behaviors and functions of the system:

1. Secure User Authentication & Authorization:

The system must allow users to register and create accounts using email and strong passwords.

The system must implement secure login mechanisms using hashed passwords (e.g., Bcrypt) and session management tokens.

2. Target Input & Validation:

The user interface must provide a dedicated field for inputting target URLs or IP addresses.

The system must validate the input format (Checking for valid URL syntax or IP structure) before initiating any scan to prevent injection attacks or errors.

3. Integrated Scanning Engine Orchestration:

The backend (Python Controller) must effectively orchestrate the execution of security tools.

It should trigger **Nmap** for port scanning, **Burp Suite** for web vulnerabilities and **OpenVAS** for infrastructure auditing sequentially or in parallel based on the scan configuration.

4. AI-Driven Analysis & Remediation:

The system must capture raw output from scanning tools and pass it to the AI Remediation Module.

The AI model must analyze the vulnerability context and generate specific, syntax-correct code patches (e.g., secure PHP/SQL code snippets) to fix the detected issues.

5. Comprehensive Reporting System:

The system must aggregate findings from all tools into a unified format.

It must generate a downloadable PDF report containing: Executive Summary, Risk Score (CVSS), Technical Evidence, and AI-Suggested Fixes.

6. Scanning History & Dashboard:

The system must maintain a history log of all previous scans stored in the database.

Users should be able to view past reports, compare results, and track the security posture improvement over time via the dashboard.

B. Non-Functional Requirements These requirements address system quality, performance, and constraints:

- **Performance & Latency:**

The web interface should load within 2 seconds under normal network conditions.

The scanning engine should process results and generate the AI report within a reasonable time frame (depending on target size) without causing browser timeouts.

- **Accuracy & Reliability:**

The system should minimize "False Positives" by cross-referencing data where possible.

The system must be reliable, handling errors gracefully (e.g., if a target is unreachable) without crashing the server.

- **Usability & UX:**

The React.js frontend must offer an intuitive, responsive design compatible with major web browsers (Chrome, Firefox, Edge).

The report layout must be readable for both technical and non-technical stakeholders.

- **Scalability:**

The system architecture must be modular, allowing for the easy addition of new scanning tools or upgrading the AI model without rewriting the core logic.

- **Data Security:**

User data (credentials, scan history) must be stored securely in the database.

Communication between the client and server should be encrypted using HTTPS protocols.

- **Availability:**

The system should be available and accessible to users 24/7 with minimal downtime.

The system should automatically recover from temporary failures whenever possible.

- **Maintainability:**

The source code should be well-structured, documented, and easy to maintain.

Future developers should be able to update system components without affecting other modules.

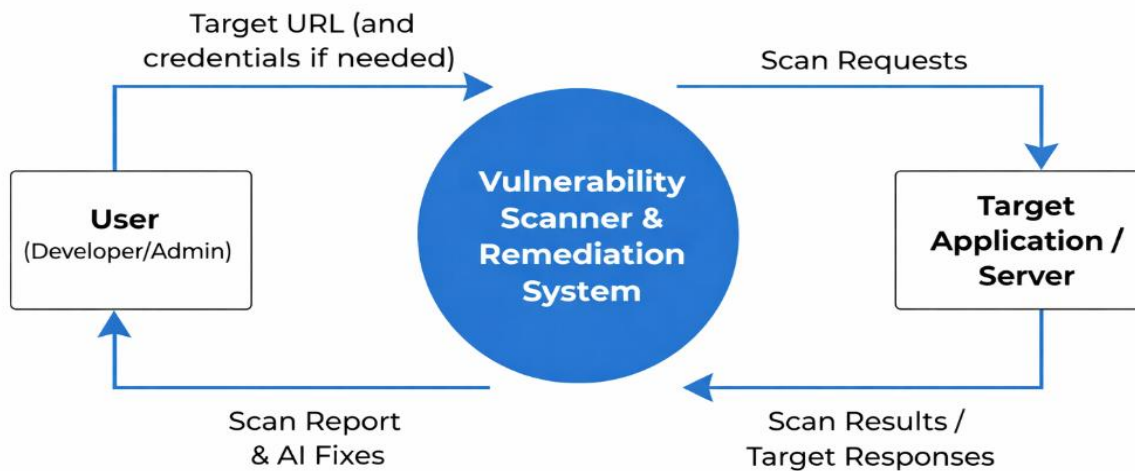
- **Backup & Recovery:**

The database should be backed up regularly to prevent data loss.

The system should support data recovery in case of hardware or software failures.

3.5 Data Flow Diagrams (DFD)

The Data Flow Diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles (processes), circles (data stores), and arrows (data flow) and squares (external entities) to show data inputs, outputs, storage points, and the routes between each destination.



Context Data Flow Diagram (Level 0)

Figure 3.1: Context Data Flow Diagram (DFD)

Diagram Description:

Figure 3.1 illustrates the context Data Flow Diagram (DFD Level 0) of the Vulnerability Scanner and Remediation System. The diagram provides a high-level overview of the system and its interaction with external entities.

The user (Developer/Admin) submits the target URL or IP address, along with credentials if required, to the system. The system processes this input and sends scan requests to the target application or server. After receiving scan results and target responses, the system analyzes the collected data to identify potential vulnerabilities.

Finally, the system generates a comprehensive vulnerability report that includes risk assessment details and AI-based remediation recommendations, and sends them back to the user.

3.6 Use Case Diagram

The Use Case diagram identifies the interactions between the system actors (Users and External Systems) and the system functionalities. It provides a high-level view of what the system does.

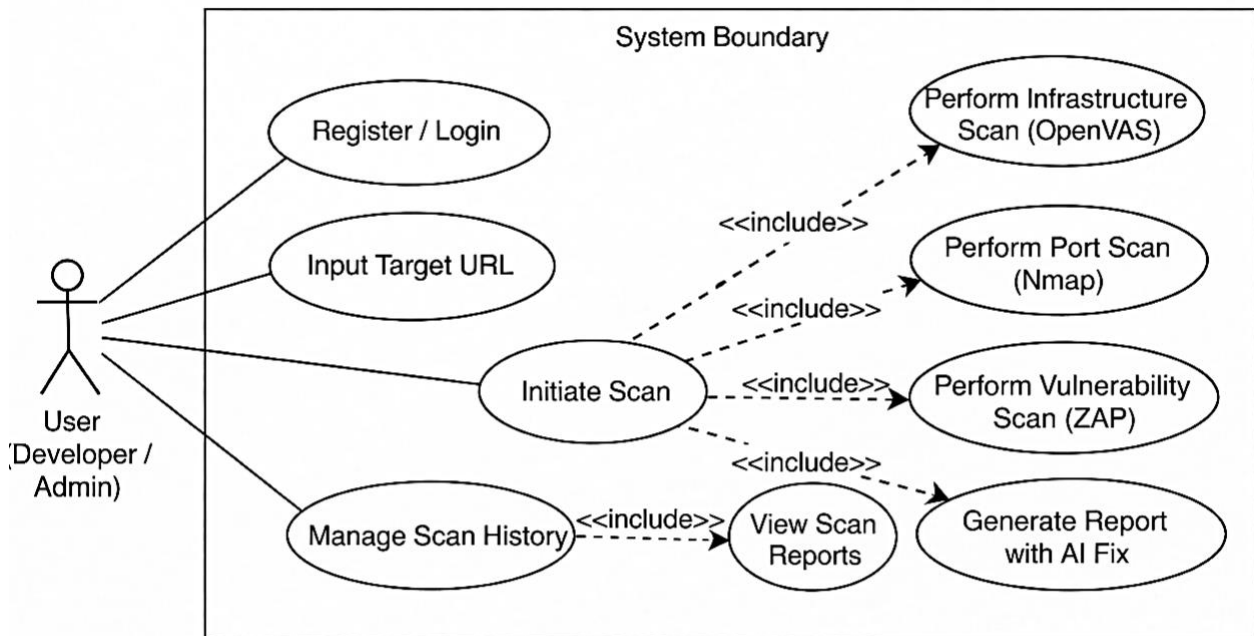


Figure 3.2: Use Case Diagram

Figure 1 3.2: Use Case Diagram

Diagram description:

Illustrates the Use case diagram of the vulnerability Scanner and remediation System.

The diagram shows the interactions between the user (Developer/Admin) and the system functionalities. The user can register or log in, input the target URL/IP address, and initiate the scanning process.

The scanning process includes network scanning using Nmap, web vulnerability scanning using OWASP ZAP, and infrastructure vulnerability assessment using openVas.

After completing the scans, the system generates a vulnerability report that includes risk assessment details and AI-based remediation recommendations. The user can also view and manage previous scan reports through the system's history logs. In addition, the system provides a user-friendly interface that simplifies the vulnerability assessment process, allowing users with different levels of technical expertise to perform security scans efficiently. The system ensures that all scan results are stored securely in the database for future reference and analysis. Furthermore, administrators can monitor scanning activities, review generated reports, and track vulnerability trends over time. This functionality helps organizations maintain continuous security awareness and improve their overall cybersecurity posture by addressing identified vulnerabilities in a timely manner.

Chapter Four: System Design

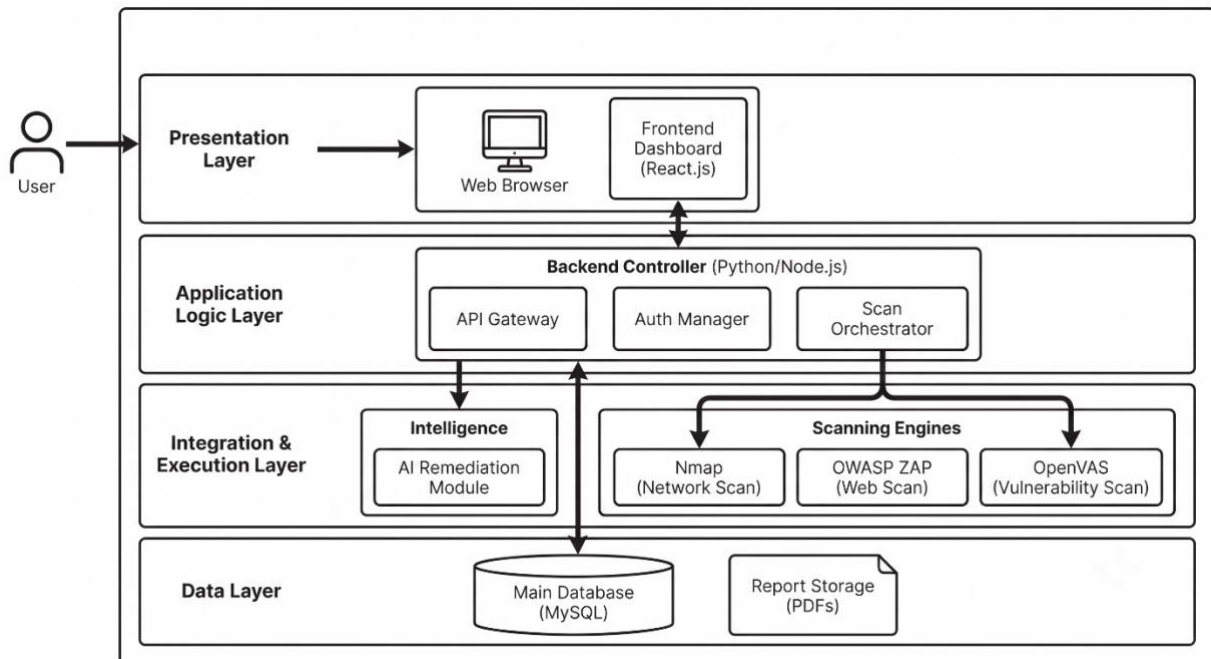


Figure 4.1: System Architecture Diagram

Diagram description:

The proposed system adopts a multi-layered architecture to provide an intelligent vulnerability scanning platform with a clear separation of responsibilities. The presentation layer offers a web-based dashboard that allows users to submit target URLs and view scan results. The Application Logic Layer manages request handling, authentication, and scan coordination through the API Gateway and scan Orchestrator.

The Integration and execution Layer performs security assessments using tools such as Nmap, OWASP ZAP and OpenVas while an AI-based remediation Module analyzes scan output and suggests mitigation actions. Finally, the Data layer ensures persistent storage of user data, scan results and generated reports, supporting structured data management and system reliability. Furthermore, the architecture is designed to be modular and scalable, allowing new security scanning tools and analysis modules to be integrated with minimal changes to the existing system structure. The separation of layers enhances maintainability by ensuring that each component performs a specific responsibility independently from other layers.

The API Gateway serves as a central communication point between the frontend interface and backend services, ensuring efficient request processing and secure data exchange. The Authentication Manager is responsible for user verification and access control, helping protect sensitive scan information and system resources from unauthorized access.

4.2 Entity Relationship Diagram (ERD)

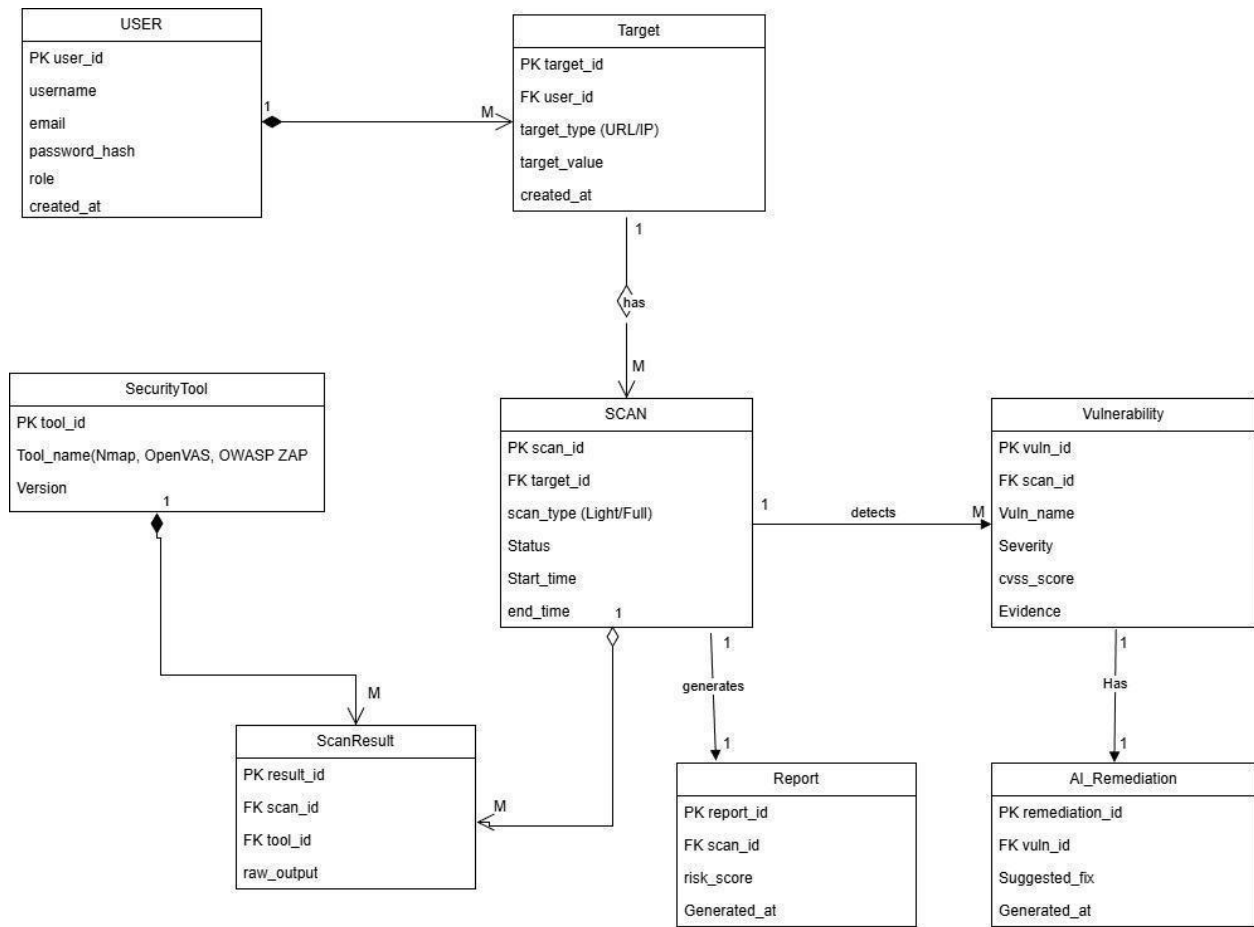


Figure 4.2: Entity Relationship Diagram (ERD)

Diagram Description:

This Entity Relationship Diagram (ERD) represents the database design of a vulnerability scanning system. The system allows users to define scan targets, perform security scans using multiple security tools, detect vulnerabilities, generate automated remediation suggestions, and produce detailed security reports. The design supports scalability, multiple scanning tools, and structured vulnerability management.

Each user can manage multiple targets, where a target may represent either a URL or an IP address. Every target can have multiple scan sessions, enabling continuous monitoring and historical tracking of security assessments. The SCAN entity acts as the core component of the system by storing scan information such as scan type, execution status, start time, and completion time.

The system integrates various security tools such as Nmap, OpenVAS, and OWASP ZAP, allowing scan results from different sources to be collected and stored in a unified format. Detected vulnerabilities are linked to their corresponding scans and include detailed information such as severity level, CVSS score, and supporting evidence for validation and analysis.

4.3 Class Diagram

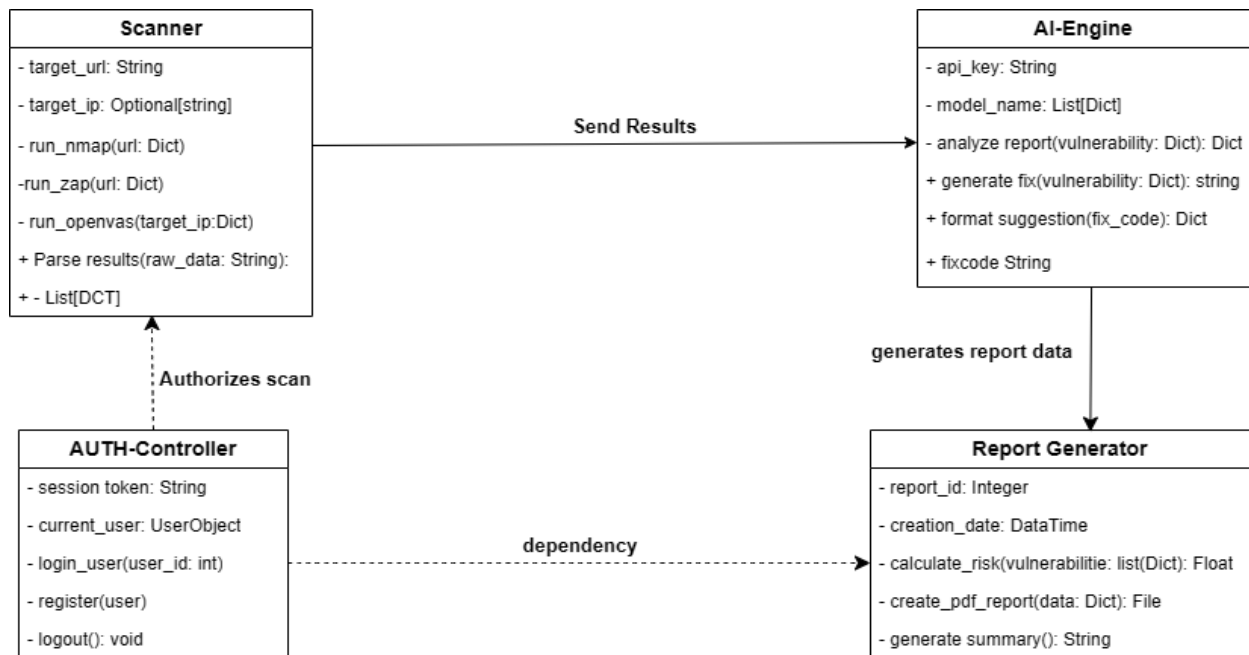


Figure 4.3: Class Diagram

Diagram Description:

This class diagram represents the structural design of the proposed vulnerability scanning and remediation system, highlighting the main classes and their interactions within the system architecture.

- **Scanner:** Responsible for executing security scanning tools such as Nmap, OWASP ZAP, and OpenVAS. It collects, parses, and organizes the scan results for further analysis.
- **AI Engine:** Processes the detected vulnerabilities and generates remediation suggestions based on predefined models, heuristics, and analysis logic.
- **Report Generator:** Creates structured reports from the analyzed results, including risk summaries and PDF outputs for stakeholders.
- **Authentication Controller:** Manages user authentication, sessions, and authorization to ensure secure access to the system.

Class Relationships:

The diagram shows the workflow of the system: scan results produced by the **Scanner** are sent to the **AI Engine** for analysis. The processed insights are then passed to the **Report Generator** to produce actionable security reports. The **Authentication Controller** ensures that only authorized users can access and interact with these system components.

4.4 Sequence Diagram

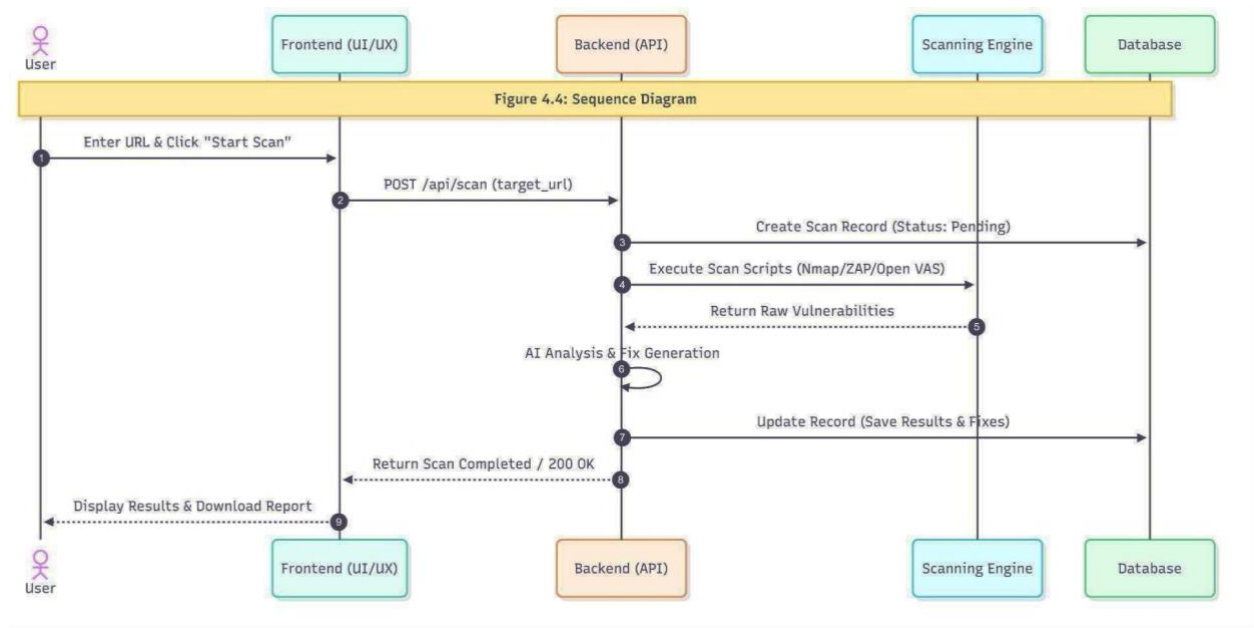


Figure 4.4: Sequence Diagram

Diagram Description:

This sequence diagram illustrates the structured interaction between the user, frontend, backend, scanning engine, AI module, and database during the penetration testing process. It highlights how the system manages data flow securely and efficiently from scan initiation to final report generation. The workflow begins when the user submits a target URL or IP address through the frontend interface. The frontend validates the input to ensure correctness and then sends a request to the backend API. Upon receiving the request, the backend creates a new scan record in the database with an initial status such as “Pending” or “In Progress.”

The backend then activates the scanning engine, which performs vulnerability assessments using security tools like Nmap, OWASP ZAP, and OpenVAS. These tools analyze the target system and generate raw security findings. Once the scanning phase is completed, the results are returned to the backend for processing. The backend forwards the findings to the AI module, which evaluates detected vulnerabilities, determines their severity levels, and generates intelligent remediation recommendations to assist in risk mitigation.

Finally, all scan results, vulnerability details, and recommended fixes are securely stored in the database. The backend then sends a completion response to the frontend, where the user can review the findings and download a comprehensive report summarizing identified vulnerabilities and suggested mitigation strategies.

This end-to-end workflow ensures automation, secure data handling, and enhanced decision-making through AI-driven security insights.

4.5 Activity Diagram

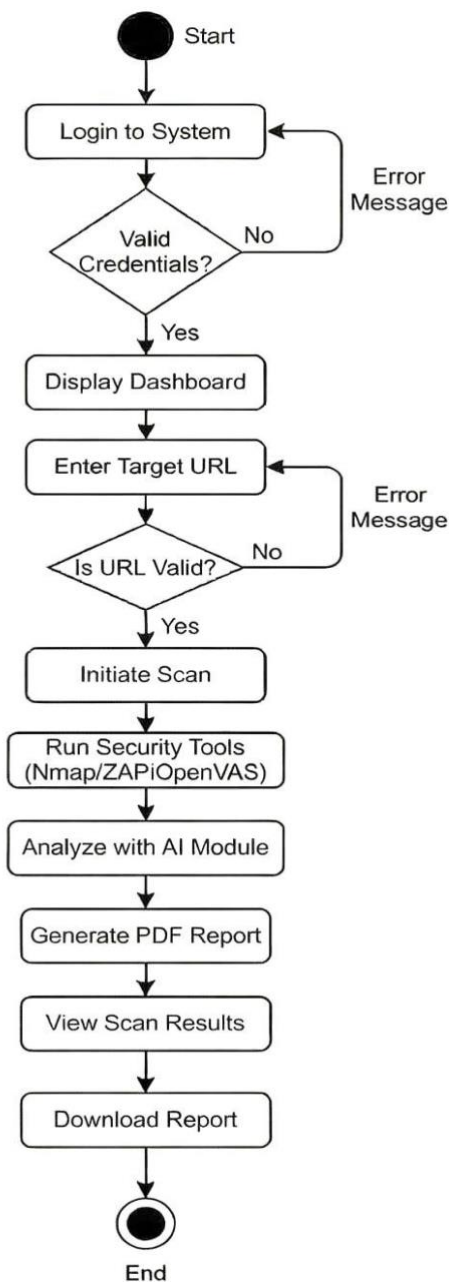


Figure 4.5: Activity Diagram

Diagram Description:

This activity diagram illustrates the main workflow of the proposed penetration testing system. The process starts with user login and credential validation, followed by entering and validating the target URL. After that the system initiates the scan, runs security testing tools, and analyze the results using an AI module. Finally, a PDF report is generated, allowing the user to view and download the scan results before the process ends.

Chapter Five: System Implementation

5.1 Used programs

- **PHP:**
A server-side scripting language used to develop the core functionality of the website, process user requests, and integrate different system components.
- **HTML:**
Used to create the structure and layout of web pages, including forms, buttons, and content organization.
- **Nmap:**
A network-scanning tool used to discover open ports, identify running services, and detect potential security weaknesses.
- **CSS:**
Used to design and style the website interface, improving the visual appearance and user experience.
- **SQL:**
Used for managing and querying the database to store, retrieve, and update system data efficiently.
- **OpenVAS:**
An open-source vulnerability assessment tool used to scan target systems and identify known security vulnerabilities.
- **Burp Suite:**
A web application security-testing tool used to analyze HTTP/HTTPS traffic and detect vulnerabilities in web applications.
- **AI API:**
Integrated to enhance vulnerability analysis and assist in generating intelligent recommendations and security reports.
- **PHPMailer:**
A PHP library used to send emails from the system, such as vulnerability reports and notifications to users.
- **XAMPP:**
A local development environment that provides Apache, PHP, and MySQL services for developing and testing the application.
- **MySQL:**
A relational database management system used to store user information, scan results, and generated reports.
- **PowerShell:**
Used to automate system tasks and execute scripts required for scanning and managing security assessment processes.
- **Visual Studio Code (VS Code):**
A source code editor used for writing, editing, debugging, and managing the project's codebase efficiently.

5.2 Screenshots of system

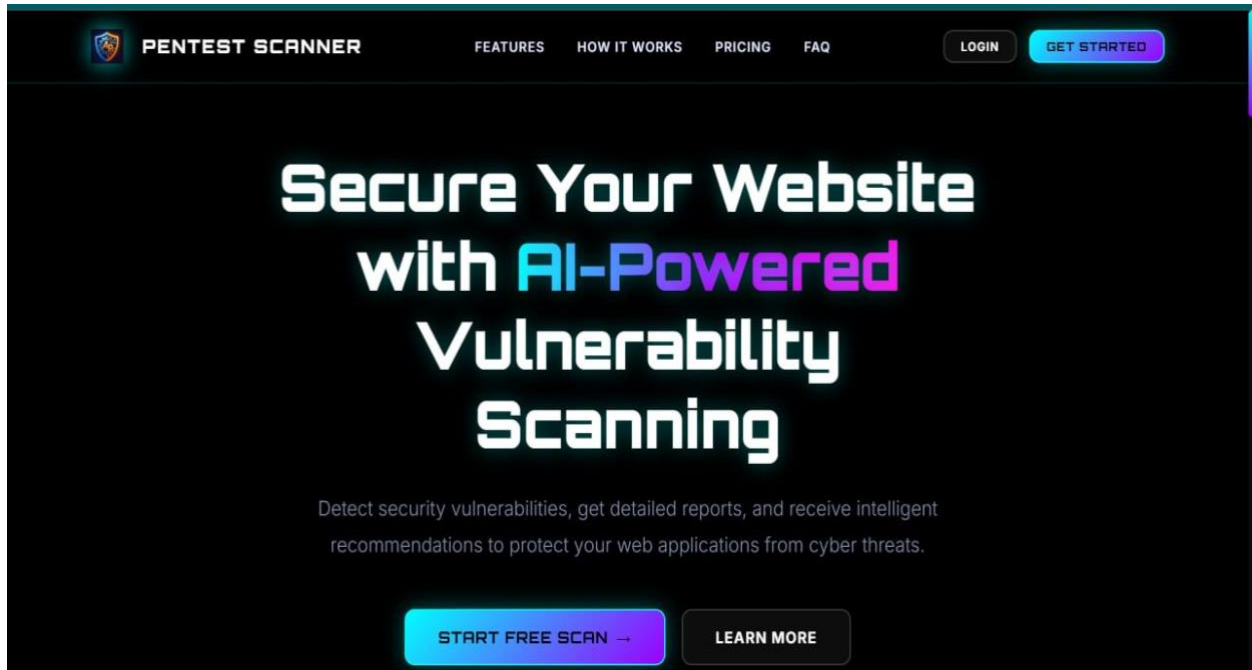


Figure 5.1: Home page

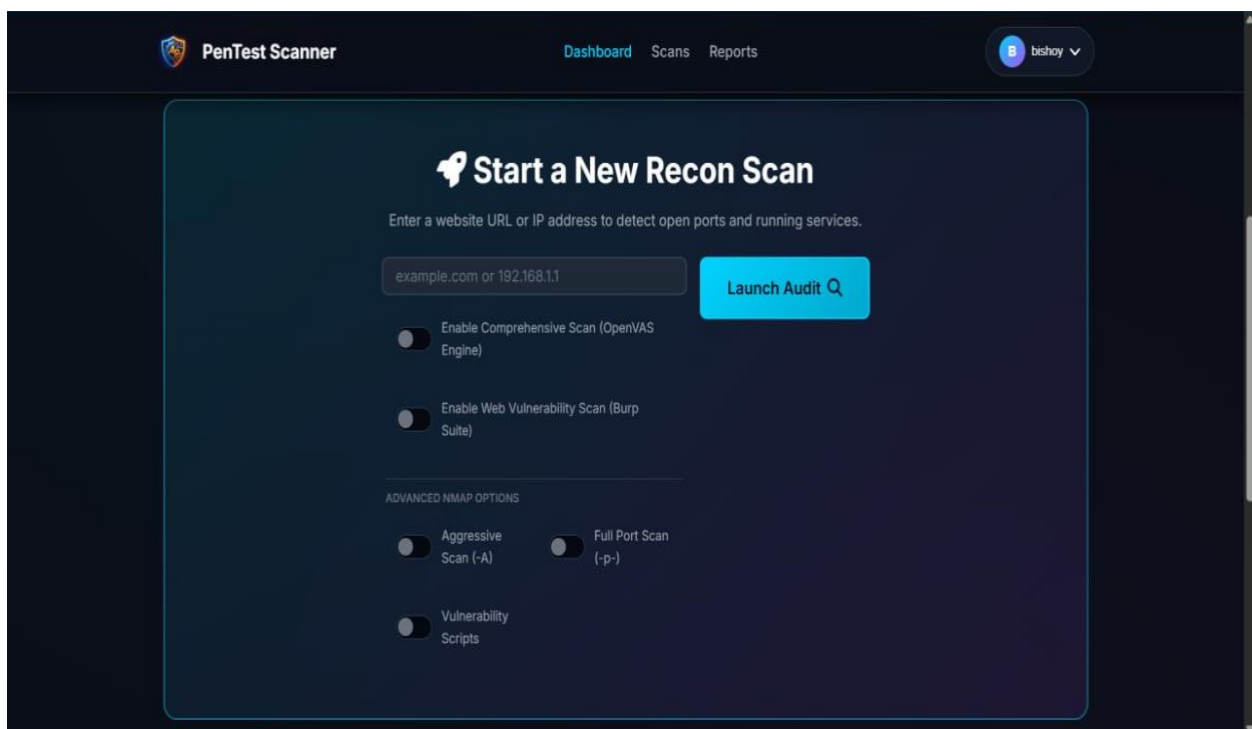


Figure 5.2: Entry page

VULNERABILITY REPORT

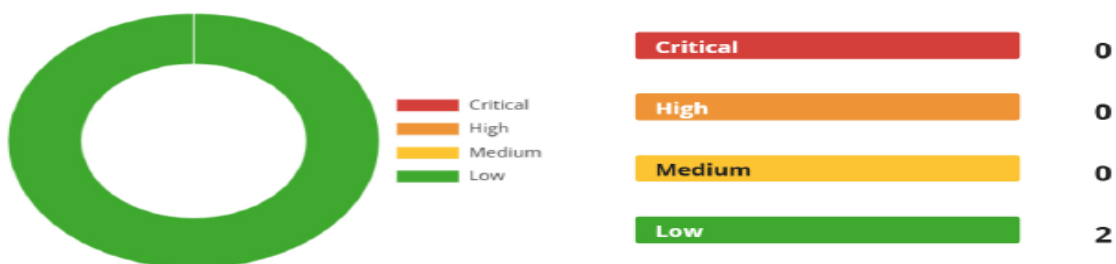
Generated by PenTest Scanner

Report Date: June 9, 2026, 10:40 pm

Scan ID: V-00090

Target Information	http://scanme.nmap.org
Scan Start Date	2026-06-09 22:39:09
Scan Duration	33 seconds
Scanner Engines	Nmap + AI Classification & RAG

Vulnerabilities by Severity



Vulnerability Details

Open Port	ID: 1
RISK FACTOR Low	
DESCRIPTION Open Port: 22/tcp (Service: ssh - OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13)	
REMEDIATION / SOLUTION Review open port 22. Ensure the service (ssh) is secure and up to date. Close the port via firewall if it is not needed publicly.	

Open Port	ID: 2
RISK FACTOR Low	
DESCRIPTION Open Port: 80/tcp (Service: http - Apache httpd 2.4.7)	
REMEDIATION / SOLUTION Review open port 80. Ensure the service (http) is secure and up to date. Close the port via firewall if it is not needed publicly.	

Figure 5.3: Vulnerability Report

5.3 Codes of the important project modules



Figure 5.4: URL validation module

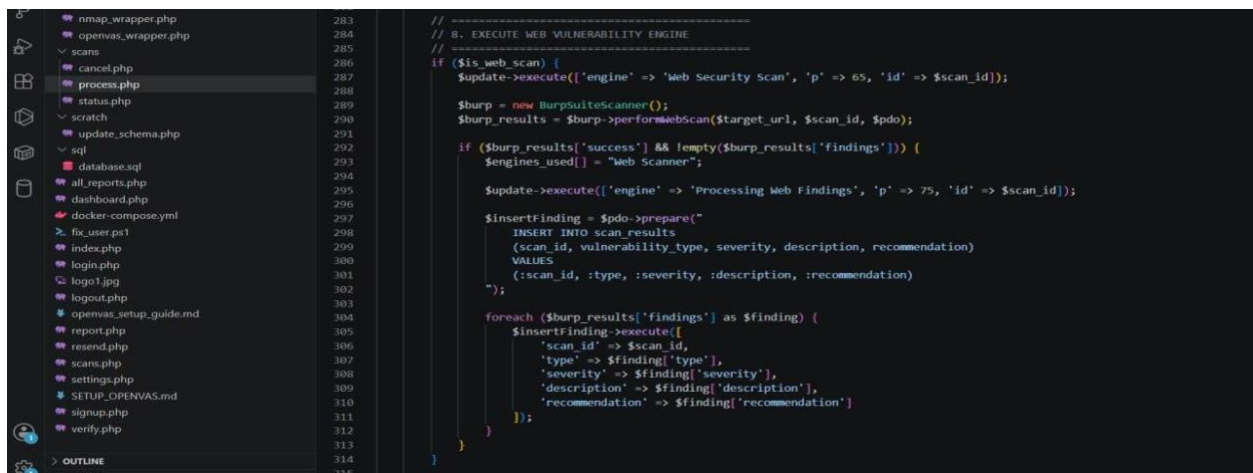
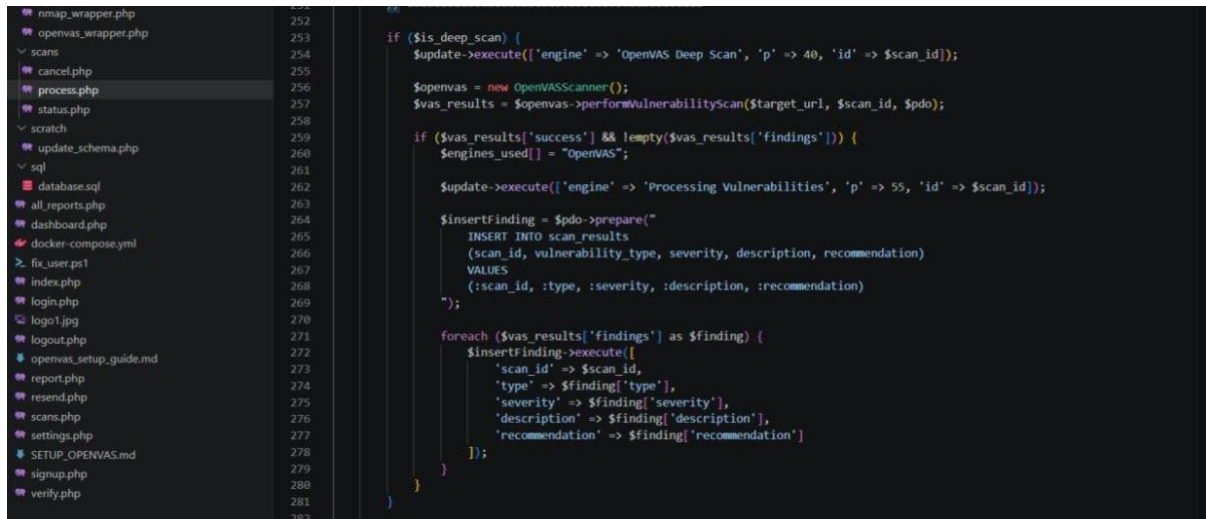
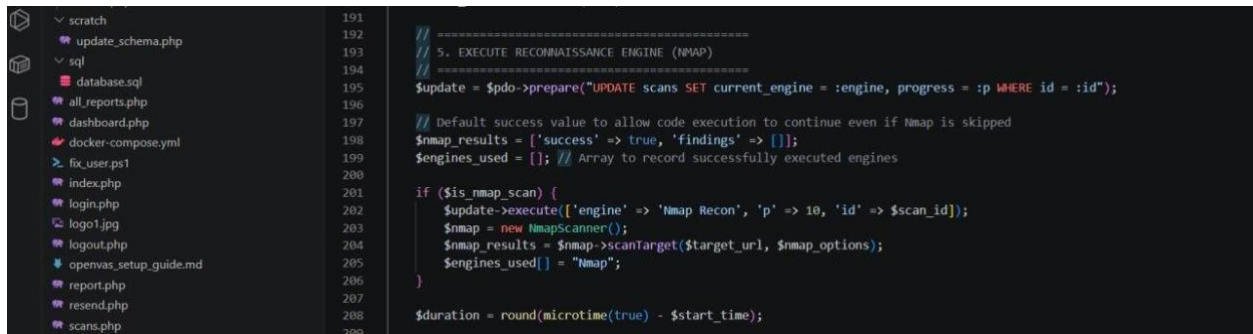


Figure 5.5: Vulnerability Scanner Module

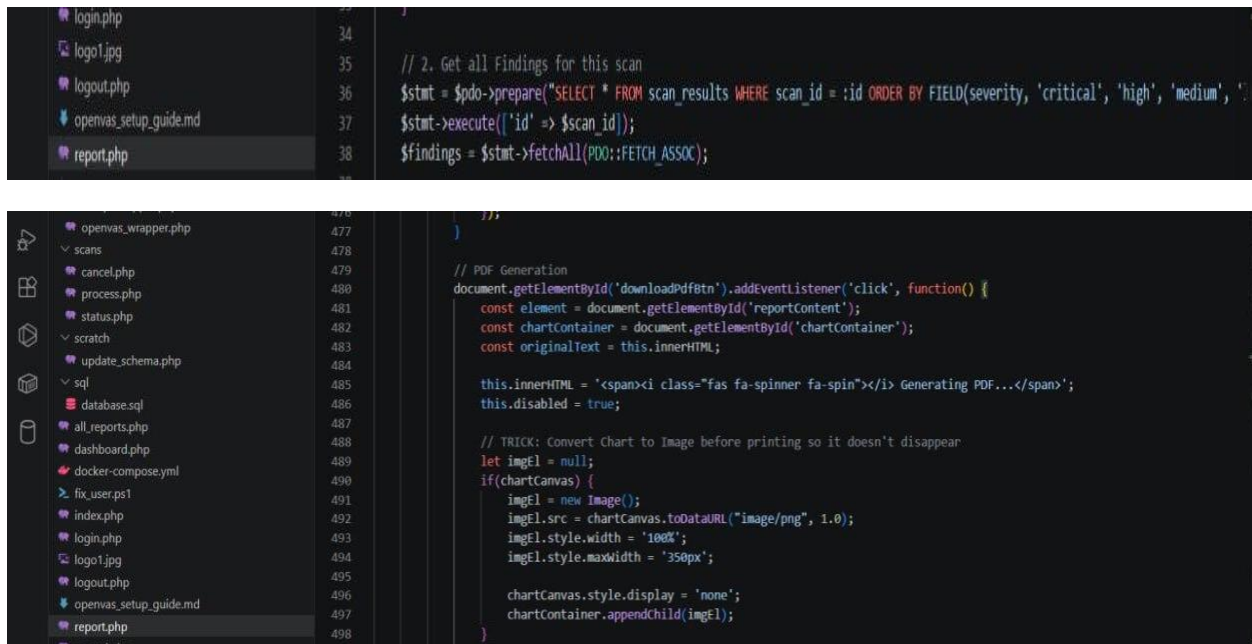


Figure 5.6: Report Generation Module

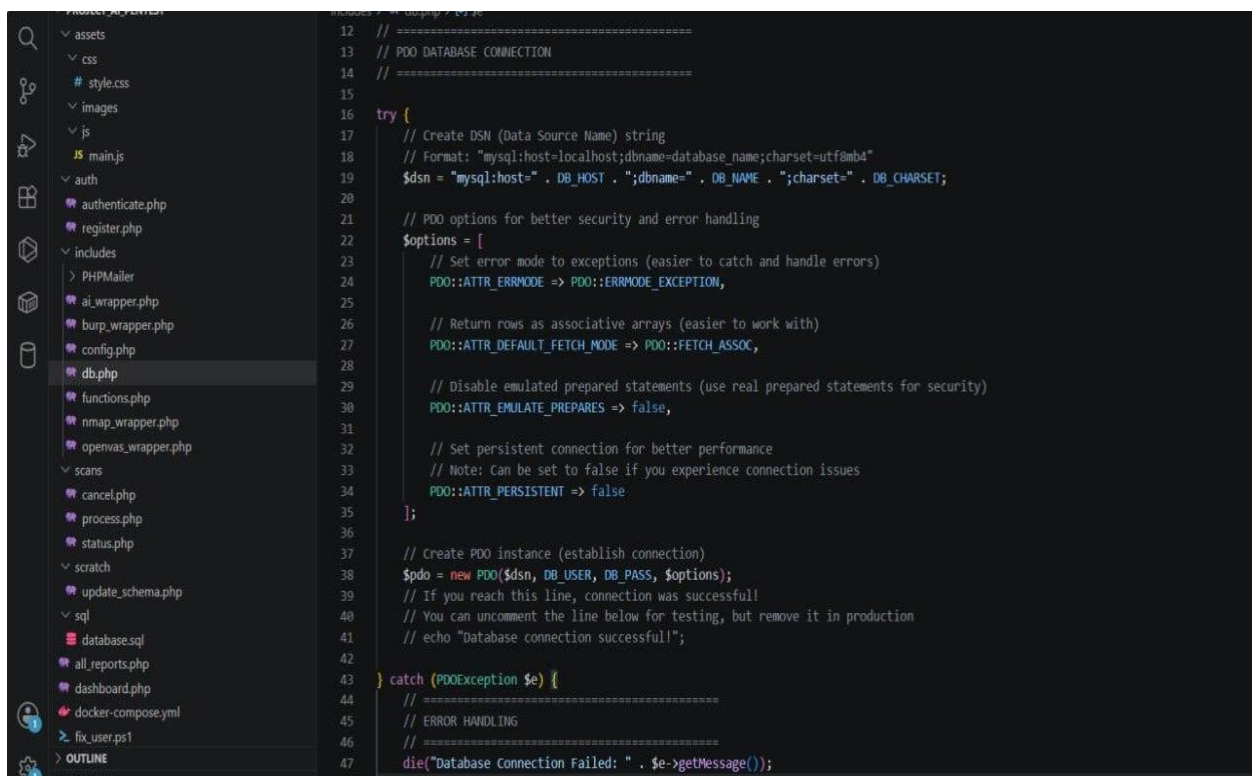


Figure 5.7: Database Connection Module

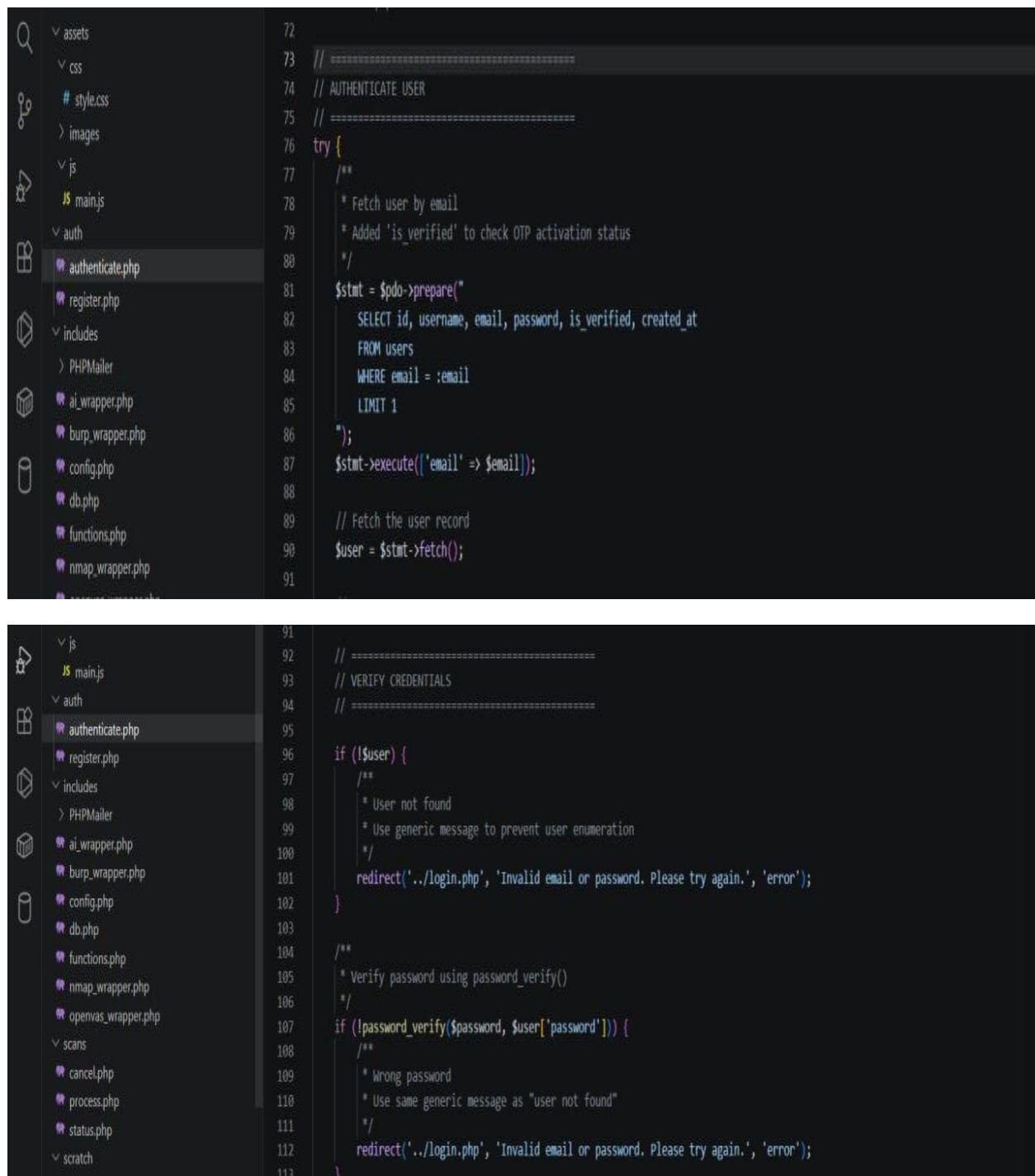


Figure 5.8: Authentication

Chapter Six: System Testing

In this chapter, the testing methodology used to ensure the quality and efficiency of the Penetration Testing System is presented. The purpose of these tests is to verify that the system is free from programming errors and capable of meeting user requirements.

6.1 Types of Testing

To ensure that the Penetration Testing System functions correctly and achieves its intended objectives, several types of testing were performed during the development process.

6.1.1 Unit Testing

Unit testing was conducted to verify that each component of the system functions correctly when tested independently.

The tested components included:

- User registration and login functionality.
- Sending the Authentication Code via email.
- URL validation.
- Vulnerability scanning process.
- Security report generation.

The results showed that each module operated correctly and produced the expected outputs.

6.1.2 Integration Testing

Integration testing was performed to ensure that all system components work together properly without any issues.

The following integrations were tested:

- Communication between the website interface and the backend server.
- Connection between PHP and the MySQL database.
- Integration with security scanning tools such as Nmap, Burp Suite, and OpenVAS.
- Integration with the AI API for analyzing scan results.
- Use of PHPMailer for sending Authentication Codes via email.

The results confirmed successful communication and data exchange between all system components.

6.1.3 Acceptance Testing

Acceptance testing was conducted to verify that the system meets the project requirements and satisfies user needs.

Several test scenarios were performed, including:

- Successfully receiving the Authentication Code and activating the account.
- Entering a valid website URL and starting the scanning process.
- Displaying detected vulnerabilities along with their severity levels.
- Generating a complete security report containing the scan results.
- Providing recommendations for vulnerability remediation.

The results demonstrated that the system functions as intended and successfully achieves its primary objective of identifying security vulnerabilities and providing reports that help improve website security.

Table 6.1: System Testing Results

Test Case	Expected Result	Actual Result	Status
User Registration	User account is created successfully	Account created successfully	Pass
User Login	User logs in successfully	Login completed successfully	Pass
Authentication Code Sending	Authentication code is sent to the user's email	Code sent successfully	Pass
URL Validation	Accept valid URLs and reject invalid URLs	Validation performed correctly	Pass
Vulnerability Scanning	Scan starts and detects vulnerabilities	Scan executed successfully	Pass
Vulnerability Display	Detected vulnerabilities are displayed correctly	Vulnerabilities displayed successfully	Pass
Report Generation	Security report is generated correctly	Report generated successfully	Pass
AI Analysis	AI provides recommendations based on scan results	Recommendations generated successfully	Pass
Database Connection	Data is stored and retrieved correctly	Database operations successful	Pass
Integration with Nmap, Burp Suite, and OpenVAS	Scanning tools communicate successfully with the system	Integration completed successfully	Pass
Account Activation	User account is activated after entering the authentication code	Account activated successfully	Pass

Chapter Seven: Conclusion and Future Work

7.1 Conclusion

This project presented a web-based Penetration Testing System designed to help identify security vulnerabilities in websites and web applications. The system allows users to submit a target URL and automatically perform security assessments using multiple vulnerability scanning tools.

The developed system integrates technologies such as PHP, MySQL, Nmap, OpenVAS, Burp Suite, AI API, and PHPMailer to provide a comprehensive security testing solution. The system is capable of detecting vulnerabilities, analyzing scan results, generating security reports, and providing recommendations for remediation.

The testing results demonstrated that the system performs efficiently and successfully achieves its objectives. It provides users with valuable insights into potential security weaknesses, helping organizations improve the security of their web applications and protect their digital assets.

7.2 Future Work

Although the developed system achieves its primary objectives, several improvements can be implemented in future versions:

Table 7.1: Future Work Plan

No.	Proposed Enhancement	Description
1	Support Additional Scanning Tools	Integrate more vulnerability scanning tools and security frameworks to improve detection capabilities.
2	Enhanced AI Analysis	Improve the AI module to provide more accurate vulnerability analysis and remediation recommendations.
3	Real-Time Monitoring	Implement continuous security monitoring and real-time vulnerability detection.
4	Security Dashboard	Develop a dashboard to track previous scans, vulnerabilities, and security statistics.
5	Automated Remediation	Add features that can automatically fix or mitigate certain types of vulnerabilities.
6	Advanced Reporting	Enhance report generation with charts, risk metrics, and detailed security insights.
7	Mobile Application Testing	Extend the system to support penetration testing for mobile applications.
8	Cloud Deployment	Deploy the system on cloud platforms to improve scalability, availability, and performance.

These enhancements would increase the effectiveness, usability, and scalability of the system while providing a more comprehensive cybersecurity assessment platform.

III-References

- [1] Gordon Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*, Insecure.Com LLC, 2009.
- [2] OpenVAS Community, "OpenVAS Documentation," Greenbone Networks. Available: <https://www.openvas.org>
- [3] PortSwigger Ltd., "Burp Suite Documentation," Available: <https://portswigger.net/burp/documentation>
- [4] OWASP Foundation, *OWASP Top 10 Web Application Security Risks*, 2021. Available: <https://owasp.org/www-project-top-ten/>
- [5] PHP Documentation Team, *PHP Manual*, Available: <https://www.php.net/docs.php>
- [6] Oracle Corporation, *MySQL Reference Manual*, Available: <https://dev.mysql.com/doc/>
- [7] PHPMailer Development Team, *PHPMailer Documentation*, Available: <https://github.com/PHPMailer/PHPMailer>
- [8] Microsoft Corporation, *PowerShell Documentation*, Available: <https://learn.microsoft.com/powershell/>
- [9] Microsoft Corporation, *Visual Studio Code Documentation*, Available: <https://code.visualstudio.com/docs>
- [10] Apache Friends, *XAMPP Documentation*, Available: <https://www.apachefriends.org>
- [11] Docker Inc., *Docker Documentation*, Available: <https://docs.docker.com>
- [12] Mozilla Developer Network (MDN), *HTML Documentation*, Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [13] Mozilla Developer Network (MDN), *CSS Documentation*, Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [14] Mozilla Developer Network (MDN), *JavaScript Documentation*, Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [15] OpenAI, *OpenAI API Documentation*, Available: <https://platform.openai.com/docs>